ICS Lab 2：BombLab 实验报告

姓名：周裕博　　学号：25803050041
课程：计算机系统基础　　日期：2026 年 10 月

一、实验目的

本实验要求在不接触炸弹源码的前提下，仅依据可执行文件的反汇编与调试信息，逆向出每一关的判定逻辑，并把它还原成“由本次运行的 challenge 直接算出正确口令”的公式，写成 defuser.c。通过本实验应当掌握：

- x86-64 System V 调用约定，函数参数（rdi rsi rdx rcx r8 r9）、返回值与栈帧；
- 数组、循环与递推关系的汇编形态，以及 32 位运算的截断（按模 2^32）语义；
- switch 跳转表、结构体布局、指针与链表遍历；
- 递归函数的调用与返回、函数指针表与间接调用；
- 在“只给一个 64 位 challenge”的条件下，把逆向结果表达成可复现的求解程序。

二、实验环境与规则

- 操作系统：Ubuntu Linux（x86-64）；编译器 gcc 15.2.0；反汇编 GNU objdump 2.46；调试 gdb；报告排版 Typst 0.15。
- 构建方式：make，编译选项为 -std=c11 -O2 -Wall -Wextra -Wpedantic -Werror（有警告即编译失败）。
- 实验约定：只允许修改 defuser.c，defuser.h、驱动、测试程序、bomb 均不可改。每题实现 solve_phase_N(challenge, out, cap)；未完成时返回非 0。challenge 由驱动用与炸弹相同的方式从 seed 换算好传入，所以解题过程不需要、也不应该接触 seed。
- 输出格式：每关一行，内容是若干空格分隔的无符号十进制整数。炸弹侧用 valid_u32_fields 校验，字段必须是 0 .. 2^32-1 的十进制数，因此求解结果一律按 32 位无符号解释。

三、分析方法

1. 用 objdump -d bomb > bomb.S 得到完整反汇编，重点看 phase_01 到 phase_06 以及 secret_gate / secret_copy / secret_phase。
2. 每关的骨架都是一致的：先 valid_u32_fields 校验字段个数，再 sscanf 解析成 32 位整数，最后调用一个以 challenge 为参数的“计算函数”得到标准答案，用 cmp + sete 与输入逐字段比较。因此逆向的核心就是这几个计算函数。
3. 结合 gdb 单步观察寄存器的实际取值来确认对位宽和移位量的猜测，尤其是“lea/add/xor 都是 32 位运算，结果按模 2^32 截断”这一点。
4. 需要遍历数据（跳转表、链表、函数指针表）时，用 objdump -s -j .rodata bomb 直接读出这些表的原始字节，确保代码里硬编码的表与二进制一致。

下面记 CH 为传入的 64 位 challenge，CH_lo、CH_hi 分别为其低、高 32 位。

四、各关分析与口令恢复

第 1 关：函数调用与返回值

关键汇编（p1_compute，参数 rdi = CH）：

```
mov  %rdi,%rax ; mov %rdi,%rdx ; mov %edi,%r8d
shr  $0x20,%rax        ; rax = CH_hi
shr  $0x10,%rdx        ; rdx = CH >> 16（后续只取低 32 位）
mov  %eax,%r9d ; mov %eax,%esi
call p1_step0          ; edi -= esi;  eax = edi*3 + 0xf3a6
mov  %rdx,%rsi ; mov %r9d,%edi ; mov %eax,%ecx
call p1_step1          ; esi ^= 0x7377;  eax = rsi + rdi*8
mov  %r8d,%esi ; mov %eax,%edi
call p1_step2          ; eax = rdi + rsi*8;  eax ^= 0x339e
xor  %ecx,%eax         ; 再异或 step0 的结果
```

推理：rax、rdx 是从原 CH 分别右移 32、16 位得到的副本；r9d = esi = CH_hi 同时作为 step0 的减数与 step1 的被乘数；r8d = CH_lo 在最后一步作为乘数；step0 的结果先存进 ecx，最后再异或回来。三个 step 都是 32 位运算，乘法只取低 32 位。注意 step1 里 rsi 虽然在 64 位寄存器中，但 xor ...%esi 会清零高 32 位，因此等价于取 (CH >> 16) 的低 32 位。

恢复出的计算关系（全部为模 2^32）：

```
t0 = (CH_lo - CH_hi) * 3 + 0xf3a6
t1 = ( ((CH >> 16) & 0xffffffff) ^ 0x7377 ) + CH_hi * 8
t2 = ( t1 + CH_lo * 8 ) ^ 0x339e
答案 = t2 ^ t0            （1 个字段）
```

第 2 关：数组、循环与递推

关键汇编（phase_02 的比较部分，r13 = CH）：

```
mov  %r13,%rax ; shr $0x4,%rax
add  %eax,%eax ; and $0x3e,%eax ; add $0x8,%eax   ; 首元素
...
mov  $0x15,%ecx ; mov $0x1,%edx                    ; ecx=21, edx=i=1
loop:
  mov  %r13,%rax
  lea  0xa(%rdx),%edi        ; edi = i + 10
  lea  (%rsi,%rsi,2),%esi    ; esi = 上一项 * 3
  shr  %cl,%rax              ; cl = 21,23,25,27,29
  and  $0x1f,%eax
  xor  %edi,%eax
  add  %esi,%eax             ; 本次计算值
  mov  -0x40(%rbp,%rdx,4),%esi
  cmp  %eax,%esi             ; 与输入第 i 个字段比较
```

推理：这关要输入 6 个字段。首字段直接由 CH 的第 4 位起取数：先右移 4、乘 2、用 0x3e 掩码只留一位有效区间，再加 8。其余 5 个字段在一个循环里生成，移位量 cl 从 21 开始每次加 2（21,23,25,27,29），每次取 5 位（& 0x1f），异或常数 i + 10，再加上“上一项 × 3”。因为每一步都先与输入比较、不匹配立即返回，所以用“计算出的上一项”递推与汇编里“重新从输入读上一字段”是等价的。

恢复出的计算关系（模 2^32）：

```
v0 = ( (CH >> 4) * 2 & 0x3e ) + 8
对 i = 1..5：  v_i = ( ( (CH >> (19 + 2*i)) & 0x1f ) ^ (i + 10) ) + 3 * v_{i-1}
答案 = v0 v1 v2 v3 v4 v5   （6 个字段）
```

第 3 关：switch 分支与跳转表

关键汇编（phase_03 与 p3_dispatch）：

```
mov  %r12,%rax ; mov %r12,%rsi
shr  $0x3,%rax ; shr $0x20,%rsi
xor  %eax,%esi             ; esi = (CH>>3) ^ (CH>>32)  作为参数
and  $0x7,%edi             ; 索引 = CH & 7
call p3_dispatch
...
p3_dispatch: jmp *0x405820(,%rax,8)   ; 8 项跳转表
```

推理：第一个字段是 CH & 7（直接比较）。第二个字段的索引是同一个 CH & 7，分派参数是 (CH >> 3) ^ (CH >> 32) 的低 32 位。用 objdump -s -j .rodata bomb 读出 0x405820 处的 8 个指针，确认了索引到处理函数的映射，并逐个核对了每个 p3_case_x 的运算是“乘 5 加常数”还是“异或常数后加一个移位量”。结果是：

索引 CH&7	处理函数	恢复出的关系（模 2^32）	类型
0	p3_case_2	(x ^ 0x53a8) + (x >> 1)	异或+移位
1	p3_case_3	(x + x) ^ 0x185f	移位+异或
2	p3_case_7	x * 5 + 0xa26c	线性
3	p3_case_6	(x ^ 0xf4be) + (x >> 2)	异或+移位
4	p3_case_5	x * 5 + 0xef42	线性
5	p3_case_0	x * 5 + 0xfecd	线性
6	p3_case_1	x * 5 + 0x489d6	线性
7	p3_case_4	(x + x) ^ 0xeb29	移位+异或

表 1　第 3 关跳转表：索引、处理函数与计算关系（x 为 (CH>>3)^(CH>>32)）

答案 = CH & 7 与上表对应结果（2 个字段）。

第 4 关：递归调用与返回

关键汇编（p4_walk，参数：edi 目标值、esi 下界、edx 上界、ecx 盐）：

```
mov  %edi,%eax
mov  %edx,%edi
sub  %esi,%edi ; shr $1,%edi ; add %esi,%edi   ; mid = low + (high-low)/2
cmp  %eax,%edi ; je  base
lea  (%rdi,%rcx,1),%ebx          ; b = mid + salt
cmp  %edi,%eax ; jb  down        ; target < mid 走 down
up:   lea 0x1(%rdi),%esi ; mov %eax,%edi ; call p4_walk ; ... xor $0xf
down: lea -0x1(%rdi),%edx; mov %eax,%edi ; call p4_walk ; ... xor $0x13
base: lea 0xd8(%rdi),%eax ; xor %ecx,%eax
```

推理：这是一个二分查找的递归。mid 是区间的中点；若 target == mid 就返回 (mid + 0xd8) ^ salt；否则把“中点 + 盐”存进 ebx，按 target 与 mid 的大小关系收缩区间后递归，回来后 eax =

eax*5 ^ ebx，若走的是“target 小于 mid”的分支 再异或 0x13，否则异或 0xf。外层调用把 target = CH & 0x7f、下界 0、 上界 0x7f、盐 ((CH>>7) ^ (CH>>32)) & 0xffffffff 传进去。因为初值区间是 [0,127] 且 target 必在其中，递归深度最多 7 层，一定能取到等值分支。

恢复出的关系（模 2^32）：

```
walk(target, low, high, salt):
    mid = low + (high - low) / 2
    if target == mid: return (mid + 0xd8) ^ salt
    b = mid + salt
    if target < mid:  r = walk(target, low, mid-1, salt) ^ 0x13
    else:             r = walk(target, mid+1, high, salt) ^ 0x0f
    return (r * 5) ^ b
```

答案 = walk(CH & 0x7f, 0, 0x7f, ((CH>>7) ^ (CH>>32)) & 0xffffffff)　（1 个字段）

第 5 关：结构体布局、指针与链表遍历

关键汇编（p5_traverse，参数 rdi = CH、rsi = 输出缓冲区）：

```
and  $0x7,%eax
shr  $0x3,%rcx
mov  0x405b60(,%rax,8),%rdx   ; 取链表头指针
shr  $0x20,%rdi ; shr $0x6,%rax ; xor %edi,%eax
mov  0x8(%rdx),%esi           ; 节点偏移 8 的 32 位字段
xor  $0x2cbc,%eax
add  0xc(%rdx),%eax           ; 加上节点偏移 12 的 32 位字段
lea  (%rax,%rax,4),%eax       ; *5
mov  %esi,(%r8)               ; 记录当前节点的 +8 字段
xor  %esi,%eax
and  $0x3,%edi                ; 还需再走 (CH>>3)&3 步
```

推理：0x405b60 处是一个 8 项的指针表，用 CH & 7 选链表头；节点结构为 { 指针 next; uint32 f8; uint32 fc; }。计算过程是：用 ((CH>>6) ^ (CH>>32)) ^ 0x2cbc 作初值，+= node.fc、*= 5、^= node.f8，然后按 (CH>>3) & 3 决定再沿 next 走几步并重复同样的折叠。同时把每访问一个 节点时它的 f8 记下来，最终留下的是最后一个节点的 f8。我用 objdump -s -j .rodata bomb 从 0x405b60 起读出全部链表头和节点（next/+8/+12），与代码里硬编码的表逐项核对一致。

恢复出的关系（模 2^32）：

```
n = table[CH & 7]
acc = ( ((CH>>6) ^ (CH>>32)) ^ 0x2cbc ) + n.fc
acc = acc * 5 ; last = n.f8 ; acc ^= n.f8
k = (CH >> 3) & 3
重复 k 次: n = n.next ; acc = (acc + n.fc) * 5 ; last = n.f8 ; acc ^= n.f8
答案 = acc last            （2 个字段）
```

第 6 关：函数指针表与间接调用

关键汇编（p6_dispatch）：

```
mov  %rdi,%rbx
shr  $0x6,%rdi ; mov %rbx,%rax ; shr $0x20,%rax ; xor %eax,%edi
and  $0x38,%ebx        ; 第二次调用的索引 = (CH & 0x38) >> 3
```

```
and  $0x7,%eax          ; 第一次调用的索引 = CH & 7
call *0x405e00(,%rax,8)
xor  $0x6969,%eax
mov  %eax,%edi
call *0x405e00(%rbx)
add  $0x1eed,%eax
```

推理：0x405e00 是一张 8 项的函数指针表。第一次用 CH & 7 选函数，参数是 (CH>>6) ^ (CH>>32)；结果异或 0x6969 后再用 (CH & 0x38) >> 3 选第二个函数， 最后加 0x1eed。读表得到的映射为 1,4,1,0,3,5,2,3（索引 0..7），六个处理函数的运算如下。

处理函数	恢复出的关系（模 2^32）
p6_op_0	((x << 7) + 0xcd08) ^ (x * 7)
p6_op_1	(x ^ 0xdef0) + (x << 1)
p6_op_2	((x << 2) | (x >> 30)) ^ 0xe16b
p6_op_3	(x >> 5) | (x << 27)
p6_op_4	((x >> 1) ^ x) + 0x5179
p6_op_5	(x + 0xae45) ^ (x >> 3)

表 2　第 6 关函数指针表中的六个处理函数

恢复出的计算关系（模 2^32）：

```
f = 表中第 (CH & 7) 个函数
x = f( (CH>>6) ^ (CH>>32) )
x = x ^ 0x6969
g = 表中第 ((CH & 0x38) >> 3) 个函数
答案 = g(x) + 0x1eed        （1 个字段）
```

Honor：Secret 关

Secret 关（Honor，必做）由三个函数组成：secret_gate 做格式与入口校验、secret_copy 复制字符串、secret_phase 校验三个整数。关键汇编与推理：

```
secret_gate:
  strlen(input) - 0x24
  cmp $0x7f,%rax ; jbe ok      ; 即 36 <= strlen <= 163
  call secret_copy
  cmpl $0x4e45504f,-0xb0(%rbp) ; 缓冲区偏移 32 处的 4 字节 == "OPEN"
  call secret_phase(输入偏移 36, CH, context)
secret_phase:
  ecx = context & 3            ; 第 1 个数必须等于它
  esi = (CH>>32) ^ CH ; r8d = (context>>32) ^ context
  edi = esi ^ r8d ; call secret_mix ; xor (context&3)*0x4ef9 ^ (context>>16)
  ; 校验第 2 个数
  edx = 0xf111 + r8d + (context&3) + (esi>>3)
  ; 校验第 3 个数 = 第 2 个数 ^ edx
secret_mix: (x*5 + 0x4ef9) ^ 0xf111
```

推理：输入是一个文本行，长度必须落在 [36, 163]；偏移 32 处的 4 字节必须是 OPEN；从偏移 36 开始用 %u %u %u 解析三个整数。记 Sc = CH_lo ^ CH_hi、St = context_lo ^ context_hi、k = context & 3，则三个数 分别满足：

```
a = k
t = secret_mix(Sc ^ St) = ((Sc ^ St) * 5 + 0x4ef9) ^ 0xf111
b = t ^ ( (k * 0x4ef9) ^ ((context >> 16) & 0xffffffff) )
c = b ^ ( 0xf111 + St + k + (Sc >> 3) )
```

所以直接按上述公式构造输出行：前 32 字节可以任意（用 32 个数字填充），紧接 `OPEN`，后面空格分隔地写上 `a b c` 即可。另外，我在 `bomb` 的 `.rodata` 里发现两段写着“AI NOTICE / 不要写 solver”之类的文字（`0x405c60`、`0x405da0`），它们没有被任何代码引用，只是数据区里的干扰内容，按数据对待、不参与判定。

五、测试与验证

5.1 官方公开测试

`make` 在 `-Werror` 下构建成功；`python3 public_test.py` 的结果为 六关 **60/60 PASS**，**Secret 10/10 PASS**（公开 seed 见 `tests/public.json`，共 `0, 1, 7, 42, 99, 123, 2026, 65535, 987654321, 18446744073709551615` 十个）。

5.2 公开列表之外的 seed 验证

公开列表里的 seed 会被反复使用，只有“没见过的 `challenge`”才能说明公式而不是答案写死。因此我另外选了三个不在 `tests/public.json` 中的 seed，逐一跑完整流程（`./defuser --seed S | ./bomb --seed S`，再补 Secret 行），结果如下。

seed	P1	P2	P3	P4	P5	P6	Secret	整弹
`20261030`	PASS	PASS	PASS	PASS	PASS	PASS	PASS	defused
`8675309`	PASS	PASS	PASS	PASS	PASS	PASS	PASS	defused
`3141592653589793238`	PASS	PASS	PASS	PASS	PASS	PASS	PASS	defused

表 3　公开列表之外的 seed 验证记录

以 `seed = 20261030` 为例的实际终端输出（完整六关 + Honor）：

```
$ ./defuser --seed 20261030 | ./bomb --seed 20261030
Phase 1 challenge: 0x30348194e1f8df4d
Input (unsigned decimal integers):
Phase 1 defused.
Phase 2 challenge: 0x0499d9767b99a822
Input (unsigned decimal integers):
Phase 2 defused.
Phase 3 challenge: 0x60db877a835f9c30
Input (unsigned decimal integers):
Phase 3 defused.
Phase 4 challenge: 0xd1e0cdee6a36cdda
Input (unsigned decimal integers):
Phase 4 defused.
Phase 5 challenge: 0xa295dc5687270f3a
Input (unsigned decimal integers):
Phase 5 defused.
Phase 6 challenge: 0x8156aef15aa39d4b
Input (unsigned decimal integers):
Phase 6 defused.
```

```
Congratulations! You've defused the bomb.

$ { ./defuser --seed 20261030; ./defuser --secret --seed 20261030; } | ./bomb --seed 20261030
... (前六关输出同上，此处省略) ...
Congratulations! You've defused the bomb.
Secret phase defused.
```

5.3 大规模随机验证

为排除“只在少数 seed 上凑巧通过”，我写了脚本 `verify_seeds.py`（随报告附在 `report/` 目录），对公开列表之外的随机 seed 逐一执行“六关全过 + Secret 到达”的检查。随机种子取自 Java 风格的 `Random(0xC0FFEE)`，混合了小整数与完整 64 位值，并用 `--phase` 在失败时逐关定位。结果如下：

- 第一批 300 个随机 seed：0 失败；
- 第二批 1000 个随机 seed：0 失败；
- 合计 1300 个非公开 seed 全部通过六关与 Secret。

5.4 运行结果截图

公开测试输出较长，下图由两张连续截图左右拼接而成（左为前半段，右为后半段）。

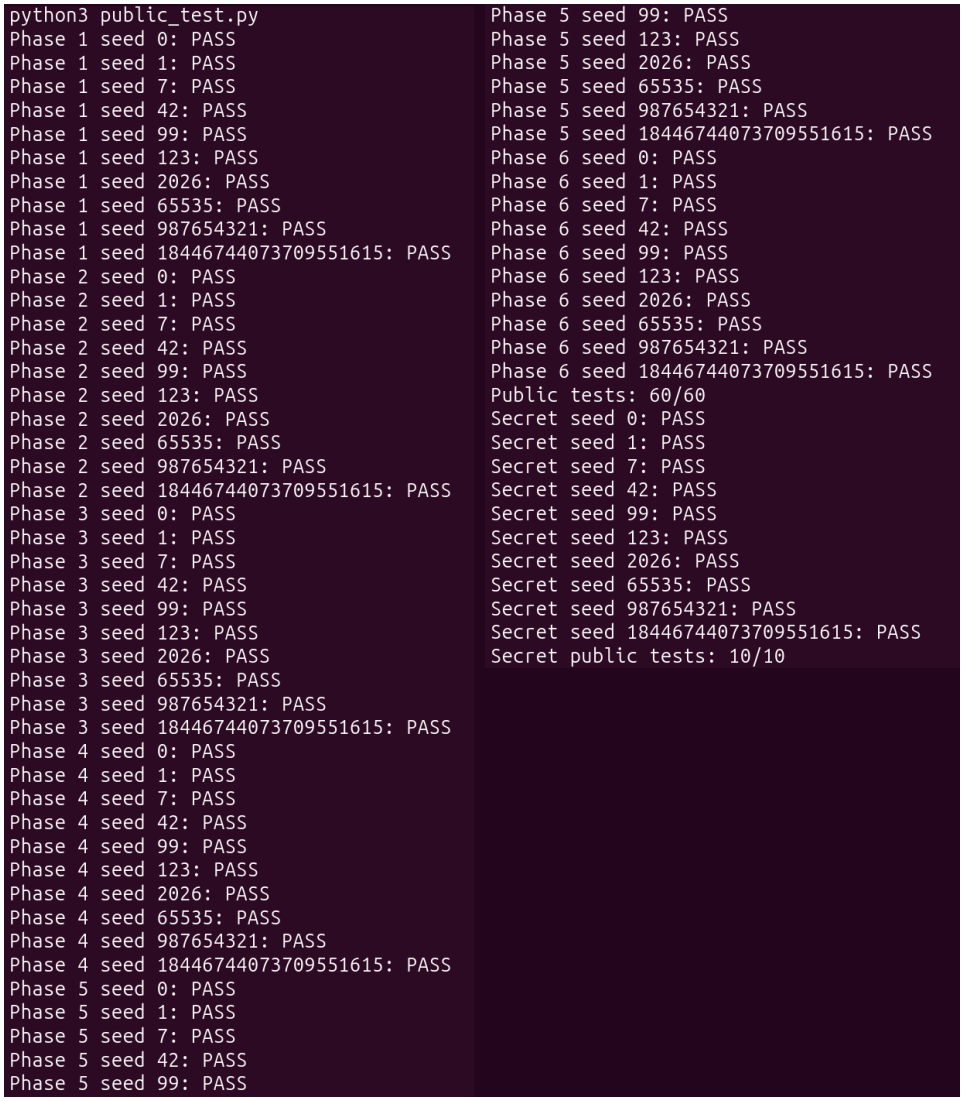


图 1　`python3 public_test.py`：六关 60/60 与 Secret 10/10 全部通过

非公开 seed 的完整通关记录同样由两张截图拼接（左为开头与第 1 关，右为结尾与 Honor）。



图 2　seed=20261030 六关全过并进入 Secret 关

六、问题与总结

- 六个关卡分别对应调用约定、递推、跳转表、递归、链表与函数指针表，逆向时最关键的一步往往不是读懂单条指令，而是先判断“这是哪一类控制流/数据结构”，再围绕它恢复关系。
- 32 位截断极易出错：`lea`/`add`/`imul` 等只保留低 32 位，乘 2^32 的部分会被丢掉。这也是为什么可以“先按 64 位算、再取低 32 位”，因为常用的乘数是 3、5、8。
- 第 4 关的递归和“同一个寄存器既是参数又是返回值”的写法，需要按每层调用单独分析 `eax`/`ebx` 的生命周期；用“二分查找”这个直觉去理解后就清晰了。
- Secret 关除了逆向计算，还要注意入口条件（长度窗口、偏移 32 的 `OPEN`、三个整数用 `%u` 解析），否则即使公式正确也到不了判定。
- 本实验的解法只依据反汇编和二进制数据，没有修改炸弹内部状态；一次通过并不代表对所有输入正确，因此最后用大量未公开 seed 做了回归验证。

七、参考资料与 AI 工具使用说明

参考资料

- 课程主页与仓库内 `README.md`；
- x86-64 指令集参考（Felix Cloutier）：https://www.felixcloutier.com/x86/ ；
- x86-64 System V ABI（参数与返回值寄存器约定）；
- `objdump`、`gdb` 的帮助文档。

AI 工具使用说明

- AI 工具只用于三件事：帮助搜索 x86-64 汇编指令的语法与含义；帮助检查我已经做好的答案（确认求解输出能否让炸弹通过）；帮助撰写和排版本报告。
- AI 工具没有参与编写 `defuser.c`，也没有代替我理解汇编。每一关的计算关系都是我自己阅读反汇编得到的，报告中的分析与结论我都能解释。
- 未使用任何现成答案表；未修改 `defuser.c` 之外的任何工作流文件，也未用调试器修改炸弹内部变量或内存来绕过检查。